

Polynomial Evaluation in the Boolean-Kernel Basis on Modern Processors

Ali Mkhida
Algorizk Labs

Abstract

Polynomial evaluation is a basic operation in computer algebra and in several polynomial-based proof systems. Its cost is commonly described by the number of arithmetic operations, although on current processors the dependency structure of the computation, memory traffic, and modular reduction also affect performance. In this article, we compare arbitrary single-point evaluation from three univariate representations: monomial coefficients, values on a roots-of-unity Lagrange domain, and a Boolean-kernel basis. The Boolean-kernel representation admits a binary reduction in which all folds at a fixed level are independent. We study this reduction over the Goldilocks field on an Apple M1 processor and compare reduction specialized to the Goldilocks modulus with Montgomery multiplication. At $N = 2^{20}$, the optimized Boolean-kernel evaluator takes 1.0560 ms, compared with 1.4116 ms for a blocked parallel monomial implementation in the same benchmark run, a reduction of 25.2% in wall time. Isolated Goldilocks and Montgomery multiplications are nearly indistinguishable, while the complete kernel evaluator is consistently faster with the specialized arithmetic backend. The measurements therefore indicate that the observed gain results from the interaction between the polynomial representation, its dependency graph, and the field-arithmetic implementation.

Polynomial evaluation is among the elementary operations on which more complex numerical and algebraic software is built. Classical evaluation schemes and their interaction with processor parallelism have been studied extensively; recent processor-oriented comparisons include Ewart et al. [4]. Horner’s rule gives a compact recurrence for a dense polynomial in monomial form, while divide-and-conquer schemes expose more independent operations at the price of additional work. In applications that store a polynomial by its values on a structured domain, interpolation formulas provide another route to an arbitrary evaluation point; barycentric Lagrange interpolation is a standard example [3].

The operation is also present in transparent proof systems and their polynomial subroutines. FRI, for example, operates on Reed–Solomon codewords and repeatedly reduces polynomial degree by structured folding [1], while STARK constructions combine such polynomial machinery with transparent computational-integrity protocols [2]. In these settings, a polynomial may be stored in coefficient form, as evaluations on a domain, or in another representation chosen to simplify a later protocol stage. Consequently, the cost of evaluating at a verifier-derived point depends not only on the degree but also on the representation in which the polynomial is available.

Arithmetic counts alone do not determine execution time on a modern processor. A short dependency chain can permit several independent field operations to proceed concurrently, whereas a recurrence with fewer operations may serialize the execution. The cost of the field multiplication is another important part of the comparison. For the Goldilocks prime

$$p = 2^{64} - 2^{32} + 1,$$

one may either exploit the special form of the modulus or keep field elements in Montgomery representation. These two choices have different instruction sequences and need not lead to the same ranking of polynomial evaluation methods.

In this work we consider a third representation, indexed by Boolean vectors,

$$K_y(X) = \prod_{i=0}^{m-1} (X^{2^i} + y_i), \quad y \in \{0, 1\}^m.$$

For $N = 2^m$, a polynomial of degree less than N is written

$$P(X) = \sum_{y \in \{0, 1\}^m} c_y K_y(X).$$

At a fixed point x , two coefficients whose indices differ only in bit i can be replaced by one coefficient using

$$(a, b) \mapsto x^{2^i}(a + b) + b.$$

The resulting computation is a complete binary reduction tree. It has linear work, but a dependency structure substantially different from Horner's rule.

The purpose of this study is to compare these evaluation structures on the same processor and with the same finite-field implementation. We first describe the polynomial representations and the arithmetic work associated with each evaluation method. We then examine the Goldilocks and Montgomery multiplication algorithms, describe the processor and measurement method, and finally present the benchmark results and their relation to the dependency graphs of the algorithms.

1 Polynomial Evaluation

Let

$$P(X) = a_0 + a_1X + \cdots + a_nX^n, \quad a_i \in \mathbb{F},$$

where $\mathbb{F}$ is a finite field. A large number of evaluation schemes can be used for the computation of $P(x)$, and their arithmetic complexity need not predict their performance on an out-of-order processor [4]. In this study we retain the schemes that correspond to the representations used in the benchmark: the classical Horner method, a blocked Horner method exposing parallelism, evaluation from values on a roots-of-unity domain, and the Boolean-kernel representation.

As in classical studies of polynomial evaluation, the first comparison is made in terms of arithmetic operations. Table 1 summarizes the main costs. These counts do not yet include the cost of the modular reduction used to realize a field multiplication. This point will be treated separately because the Goldilocks modulus allows more than one reasonable multiplication strategy.

Table 1: Arithmetic structure of the evaluation methods considered in this study. For the last two rows $N = 2^m$.

Method	Multiplications	Additions	Main dependency
Horner	n	n	chain of length n
Blocked Horner	$n + \theta(B)$	n	independent blocks
Lagrange, batch inversion	$O(N)$ +one inversion	$O(N)$	two linear scans
Boolean-kernel fold	$N - 1$	$2(N - 1)$	binary tree

The number of operations gives useful information, but it is not sufficient to predict the execution time. Horner's rule has the smallest and most regular operation count, but its successive multiply-add steps form a dependency chain. The blocked Horner and Boolean-kernel methods expose independent operations. The Lagrange method requires more arithmetic and temporary storage, but part of its work is also parallel. The relation between these properties and the measured execution time is one of the questions considered in this article.

1.1 Horner Method: Classical

The classical Horner method evaluates the monomial representation recursively. For an input value x ,

$$P(x) = a_0 + x(a_1 + x(a_2 + \cdots + xa_n)).$$

Starting from the highest coefficient, the recurrence is

$$r_n = a_n, \quad r_i = a_i + xr_{i+1}, \quad i = n-1, \dots, 0,$$

and $P(x) = r_0$.

The method uses n field multiplications and n field additions. Its advantage is therefore clear from the arithmetic point of view. The drawback is also immediate: r_i cannot be computed before r_{i+1} . The critical path contains all n multiplication steps. For large n , the execution of one polynomial gives little instruction-level parallelism even when the processor has several execution units available.

1.2 Horner Method: Blocked

The dependency chain can be shortened by splitting the coefficients into blocks. Let B be a block size and, for simplicity, suppose that $N = n + 1$ is divisible by B . Write

$$P(X) = \sum_{j=0}^{q-1} X^{jB} P_j(X), \quad q = \frac{N}{B},$$

where

$$P_j(X) = a_{jB} + a_{jB+1}X + \cdots + a_{jB+B-1}X^{B-1}.$$

The q polynomials $P_j(x)$ are independent and can be evaluated in parallel by the classical Horner method. Once they have been computed, the final combination is another Horner recurrence, now in

$$x^B : \quad P(x) = P_0(x) + x^B (P_1(x) + x^B (\cdots + x^B P_{q-1}(x))).$$

Ignoring the cost $\theta(B)$ required to obtain x^B , the block evaluations and the final combination together use n field multiplications and n field additions. The arithmetic count is thus close to the classical method, but the critical path is reduced because the block evaluations can proceed concurrently. In the implementation used later, the blocks are distributed between Rayon workers and the final combination is performed after the local Horner evaluations.

1.3 Lagrange Representation on a Roots-of-Unity Domain

A polynomial may also be represented by its values on a structured evaluation domain. Let

$$H = \{1, \omega, \omega^2, \dots, \omega^{N-1}\},$$

where ω is a primitive N -th root of unity, and let

$$v_i = P(\omega^i).$$

For a point $x \notin H$, the Lagrange formula reads

$$P(x) = \sum_{i=0}^{N-1} v_i L_i(x),$$

with

$$L_i(x) = \frac{(x^N - 1)\omega^i}{N(x - \omega^i)}.$$

The expression is the roots-of-unity specialization of the Lagrange interpolation formula; barycentric forms of Lagrange interpolation are reviewed in [3]. The denominators

$$x - \omega^i$$

may be inverted together by the standard batch-inversion technique. The implementation therefore replaces N independent field inversions by one field inversion and two linear multiplication scans. The remaining numerator accumulation is linear in N , and it admits a parallel reduction.

This representation is common when the surrounding computation already uses evaluations on a roots-of-unity domain. For one arbitrary off-domain point, however, the evaluation requires substantially more field arithmetic and temporary memory than Horner's rule. The benchmark in Section 4 measures this cost directly.

1.4 Boolean-Kernel Representation

We now consider the representation used in this work. Let

$$N = 2^m$$

and identify an integer

$$0 \leq y < N$$

with its binary vector

$$y = (y_0, \dots, y_{m-1}) \in \{0, 1\}^m.$$

For each such vector define

$$K_y(X) = \prod_{i=0}^{m-1} (X^{2^i} + y_i).$$

Every K_y has degree $N - 1$, and the family

$$\{K_y : y \in \{0, 1\}^m\}$$

forms a basis of the vector space of polynomials of degree smaller than N . We write

$$P(X) = \sum_{y \in \{0, 1\}^m} c_y K_y(X).$$

The basis differs from the monomial basis in an important point. Its elements share a product structure indexed by the bits of y . At a fixed point x , the factor associated with bit i is either

$$x^{2^i}$$

or

$$x^{2^i} + 1.$$

Pairs of coefficients that differ only in this bit can therefore be combined before the remaining factors are evaluated.

The change of basis from monomial coefficients can be expressed as a Boolean zeta/Möbius transform and requires $O(N \log N)$ field additions. That conversion is not included in the isolated single-point evaluation benchmark. As with the Lagrange representation, the benchmark assumes that the polynomial is already available in the representation under study.

1.5 Boolean-Kernel Evaluation by Folding

Let

$$t_i = x^{2^i}.$$

Consider two terms whose Boolean indices agree except at coordinate i . After factoring all the common kernel factors, their contribution has the form

$$a t_i + b(t_i + 1).$$

It can be replaced by the single coefficient

$$F_{t_i}(a, b) = t_i(a + b) + b.$$

Thus one Boolean coordinate is eliminated by the local transformation

$$(a, b) \mapsto t_i(a + b) + b.$$

Starting from the vector

$$(c_0, c_1, \dots, c_{N-1}),$$

the first level combines adjacent pairs using $t_0 = x$. The next level combines adjacent results using $t_1 = x^2$, and the process continues with

$$t_{i+1} = t_i^2$$

until one field element remains. The final value is $P(x)$.

At level i , the number of folds is

$$\frac{N}{2^{i+1}}.$$

The total number is therefore

$$\frac{N}{2} + \frac{N}{4} + \dots + 1 = N - 1.$$

Each fold contains one field multiplication and two field additions. The evaluation consequently uses

$$N - 1$$

multiplications,

$$2(N - 1)$$

additions, and

$$m - 1$$

squarings for the sequence $t_0, \dots, t_{m-1}$.

(a) Horner dependency chain



(b) Boolean-kernel reduction

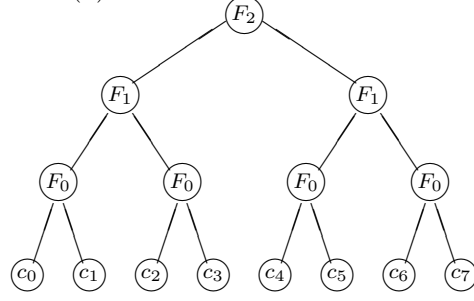


Figure 1: Dependency graphs for $N = 8$. The classical Horner method forms a single recurrence. The Boolean-kernel method has four independent folds at the first level, two at the second, and one at the last level. Here $F_i(a, b) = x^{2^i}(a + b) + b$.

Figure 1 compares the dependency graph with the classical Horner recurrence for a small instance. The Horner graph is one chain. The Boolean-kernel graph has the same logarithmic number of levels as a balanced reduction tree, with all nodes at one level independent.

The two graphs illustrate the principal difference considered in the performance study. Horner’s method performs fewer additions and has a very compact implementation, whereas the Boolean-kernel method trades additional addition work for a much shorter critical path and a large amount of independent work in the lower levels of the tree.

2 Goldilocks Field Arithmetic

The polynomial evaluation methods of Section 1 ultimately depend on the cost of field multiplication. We therefore separate the choice of polynomial representation from the choice of modular multiplication algorithm.

The experiments use the Goldilocks prime

$$p = 2^{64} - 2^{32} + 1.$$

A field element is represented by one machine word. Two multiplication strategies are considered. The first exploits the special form of the modulus. The second uses Montgomery representation with radix

$$R = 2^{64}.$$

Both strategies are retained in the experiments so that the effect of the modular multiplication algorithm can be measured independently of the polynomial representation.

2.1 Reduction Specialized to the Goldilocks Modulus

Let

$$z = ab$$

be the 128-bit product of two canonical residues

$$0 \leq a, b < p.$$

Write

$$z = z_{\text{lo}} + 2^{64}z_{\text{hi}},$$

where

$$0 \leq z_{\text{lo}}, z_{\text{hi}} < 2^{64}.$$

For the Goldilocks prime,

$$2^{64} \equiv 2^{32} - 1 \pmod{p}.$$

Define

$$\varepsilon = 2^{32} - 1.$$

Then

$$z \equiv z_{\text{lo}} + \varepsilon z_{\text{hi}} \pmod{p}.$$

A direct application of this congruence already reduces the 128-bit product to a value close to one machine word. A more useful form is obtained by splitting the high half as

$$z_{\text{hi}} = h_0 + 2^{32}h_1, \quad 0 \leq h_0, h_1 < 2^{32}.$$

Since

$$2^{32}\varepsilon = 2^{64} - 2^{32} \equiv -1 \pmod{p},$$

we obtain

$$\varepsilon z_{\text{hi}} \equiv h_0\varepsilon - h_1 \pmod{p}.$$

Therefore

$$z \equiv z_{\text{lo}} - h_1 + h_0(2^{32} - 1) \pmod{p}.$$

This expression involves only the 128-bit product ab , extraction of the two 32-bit halves of z_{hi} , shifts, additions, and subtractions, followed by a small number of corrections to return to the canonical interval.

The implementation used in the benchmark follows this reduction rather than a generic integer remainder instruction.

Algorithm 1. Goldilocks multiplication.

Input: $a, b \in [0, p)$,

Output: $ab \bmod p$.

```

 $z \leftarrow a \cdot b$  (full 128-bit product),
 $z_{\text{lo}} \leftarrow z \bmod 2^{64}$ ,
 $z_{\text{hi}} \leftarrow \lfloor z/2^{64} \rfloor$ ,
 $h_0 \leftarrow z_{\text{hi}} \bmod 2^{32}$ ,
 $h_1 \leftarrow \lfloor z_{\text{hi}}/2^{32} \rfloor$ ,
 $u \leftarrow z_{\text{lo}} - h_1 + h_0(2^{32} - 1)$ ,
 $u \leftarrow$  canonical reduction of  $u$  modulo  $p$ ,
return  $u$ .
```

The exact instruction sequence generated from this formula depends on the compiler and target architecture. For this reason the multiplication is measured separately before its effect on the complete polynomial evaluator is considered.

2.2 Montgomery Representation

Montgomery multiplication replaces division by p with arithmetic modulo a power of two [5]. Let

$$R = 2^{64}.$$

Because p is odd,

$$\gcd(p, R) = 1,$$

and R is invertible modulo p .

A canonical field element a is represented in Montgomery form by

$$\tilde{a} = aR \bmod p.$$

Multiplication of two Montgomery residues produces

$$\tilde{a}\tilde{b} = abR^2$$

and is followed by a Montgomery reduction, which removes one factor of R :

$$\text{REDC}(\tilde{a}\tilde{b}) = abR \bmod p.$$

For the modulus considered here, the constants are

$$R \bmod p = 2^{32} - 1 = \text{0x00000000ffffffff},$$

$$n' = -p^{-1} \bmod R = \text{0xffffffffffffffff},$$

and

$$R^2 \bmod p = \text{0xffffffffe00000001}.$$

The value $R^2 \bmod p$ is used to convert a canonical residue into Montgomery form with one Montgomery multiplication.

2.3 Montgomery Reduction

Let

$$0 \leq T < pR.$$

The Montgomery reduction computes

$$TR^{-1} \bmod p$$

without an integer division by p .

Using

$$n' = -p^{-1} \bmod R,$$

define

$$m = (T \bmod R)n' \bmod R.$$

By construction,

$$T + mp \equiv 0 \pmod{R},$$

so the quotient

$$u = \frac{T + mp}{R}$$

is an integer. The result satisfies

$$u \equiv TR^{-1} \pmod{p}.$$

A final conditional subtraction returns the canonical representative.

Algorithm 2. Montgomery reduction.

Input: $T, \quad 0 \leq T < pR,$
Output: $TR^{-1} \bmod p.$

$$\begin{aligned} m &\leftarrow (T \bmod R)n' \bmod R, \\ u &\leftarrow (T + mp)/R, \\ \text{if } u \geq p &\quad u \leftarrow u - p, \\ \text{return} &\quad u. \end{aligned}$$

For a 64-bit radix, the sum

$$T + mp$$

may require 129 bits. The implementation therefore records the carry from the 128-bit addition explicitly before extracting the high word.

2.4 Montgomery Multiplication

If $\tilde{a}$ and $\tilde{b}$ are already stored in Montgomery form, their product is computed as

$$\text{MontMul}(\tilde{a}, \tilde{b}) = \text{REDC}(\tilde{a}\tilde{b}).$$

Algorithm 3. Montgomery multiplication.

Input: $\tilde{a} = aR \bmod p, \quad \tilde{b} = bR \bmod p,$
Output: $abR \bmod p.$

$$\begin{aligned} T &\leftarrow \tilde{a}\tilde{b}, \\ u &\leftarrow \text{REDC}(T), \\ \text{return} &\quad u. \end{aligned}$$

Conversion from canonical to Montgomery form is performed as

$$\tilde{a} = \text{REDC}(a(R^2 \bmod p)),$$

while conversion back to the canonical representation is

$$a = \text{REDC}(\tilde{a}).$$

In the multiplication benchmark, operands are kept in their native representation throughout the timed region. Thus the specialized Goldilocks implementation receives canonical residues, while the Montgomery implementation receives Montgomery residues. Conversion is measured separately when it is relevant to an application and is not charged once per multiplication.

2.5 Kernel Fold Under the Two Arithmetic Models

The Boolean-kernel reduction uses

$$F_t(a, b) = t(a + b) + b.$$

With canonical Goldilocks residues, the implementation can combine the final addition by b with the reduction of the 128-bit product. This gives a specialized field operation rather than a literal sequence of one modular addition, one modular multiplication, and a second modular addition.

In Montgomery form the same mathematical operation is

$$\tilde{F}_t(\tilde{a}, \tilde{b}) = \text{MontMul}(\tilde{t}, \tilde{a} + \tilde{b}) + \tilde{b}.$$

All operands remain in Montgomery form, so no conversion is needed between levels of the reduction tree.

Table 2 summarizes the two multiplication strategies before measurements.

Table 2: Structure of the two Goldilocks multiplication strategies.

Method	Representation	Main reduction operation
Goldilocks special	canonical residue	use $2^{64} \equiv 2^{32} - 1 \pmod{p}$
Montgomery	$aR \bmod p$	$(T + mp)/2^{64}$ with $m = T_0 n'$

At this point there is no architectural conclusion to draw from the operation formulas alone. The specialized reduction uses the algebraic form of the Goldilocks modulus; Montgomery reduction introduces additional word multiplications but has a regular reduction structure. The generated instruction sequences and the complete evaluation times are therefore examined experimentally in the following sections.

3 Processor and Measurement Methods

3.1 Latency and Throughput Definitions

The execution time of a polynomial evaluator depends on two distinct properties of the instruction stream. The first is the delay introduced by a dependency chain. The second is the rate at which independent instructions of the same class can be issued.

A *dependency chain* is a sequence of operations in which each result is required by the next operation. The classical Horner recurrence is the simplest example in this study:

$$r_i = a_i + x r_{i+1}.$$

Even if several multiplication units are available, the next multiplication cannot begin before the preceding value r_{i+1} has been produced.

The *latency* of an operation is the delay that it contributes to such a dependency chain. The *throughput* is the rate at which independent operations can be completed when enough independent operands are available. For the algorithms considered here, neither quantity alone is sufficient. A method with a longer local operation sequence can still execute faster if many instances of that sequence are independent.

The Boolean-kernel reduction makes this distinction explicit. At level i , all

$$\frac{N}{2^{i+1}}$$

folds are independent, while the transition from one level to the next is a true dependency. The critical path therefore grows with the number of levels, $m = \log_2 N$, rather than with the number of coefficients.

3.2 Architecture Details

The measurements reported in this study were obtained on an Apple M1 processor. The comparison follows the same general processor-oriented principle used in earlier work on polynomial evaluation: arithmetic count alone is insufficient when evaluation schemes expose different dependency graphs [4]. The M1 is an out-of-order superscalar processor, so several integer operations can be in flight simultaneously when the instruction scheduler finds independent work. Horner evaluation, blocked Horner evaluation, and the Boolean-kernel tree expose very different amounts of instruction-level and task-level parallelism.

We do not attempt to model the complete microarchitecture. Only the features required to interpret the measurements are considered: independent integer multiplication, dependency depth, memory traffic, allocation, and the scheduling of independent subtrees across CPU workers.

The field implementation also matters at the instruction level. A Goldilocks multiplication specialized to

$$p = 2^{64} - 2^{32} + 1$$

uses one full-width product followed by shifts, additions, and subtractions. A Montgomery multiplication uses the initial product and an additional product involving the reduction coefficient. Figure 2 shows the corresponding dependency structure schematically.

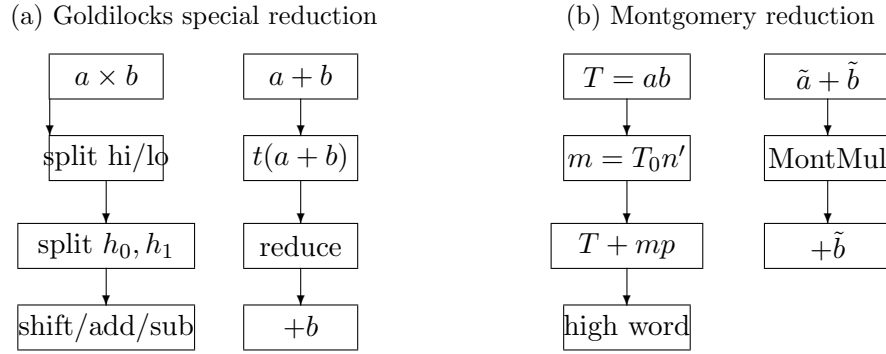


Figure 2: Schematic dependency graphs for the two field-arithmetic paths. The left side shows the specialized Goldilocks product reduction and its use in the kernel fold; the right side shows the Montgomery reduction path. The diagram represents dependencies rather than exact machine instructions.

3.3 Exact Arithmetic

Unlike floating-point polynomial evaluation, the present computation does not involve a numerical approximation error. Every benchmarked method is required to return exactly the same element of $\mathbb{F}_p$.

Correctness is checked before performance measurements. The test suite compares the optimized Goldilocks multiplication with a reference modular multiplication, compares the fused Boolean-kernel fold with its direct field expression, verifies the monomial-to-kernel change of basis by evaluating the same polynomial in both representations, and verifies the Lagrange implementation against the monomial reference.

The Montgomery implementation is subjected to the same requirement. Canonical residues are converted to Montgomery form, multiplied there, and converted back before comparison with

the reference result. This prevents a performance result from being accepted merely because two implementations use different internal encodings.

3.4 Measurement Tools

The benchmarks are implemented with Criterion. The principal measurements use a warm-up period before sampling and report an interval together with a central estimate. The benchmark harness applies `black_box` to inputs and results in order to prevent the compiler from replacing the measured computation with a constant result.

Microbenchmarks are used for the field operations and the kernel fold. Complete polynomial evaluators are measured separately. This separation is important: an improvement in the latency of one multiplication need not translate linearly to the full evaluator once memory traffic, worker scheduling, and allocation are included.

For cross-method comparisons, all implementations are compiled from the same repository revision and executed on the same machine. A ratio is formed only from measurements collected under the same field backend and benchmark configuration.

3.5 Programming Model

The implementation is written in Rust. Parallel work is expressed with Rayon. The monomial baseline divides the coefficient vector into blocks, evaluates those blocks independently, and combines the resulting values. The Boolean-kernel implementation divides the reduction tree into power-of-two subtrees. Each worker reduces one complete subtree locally, thereby avoiding a global synchronization after every tree level.

The optimized kernel evaluator additionally reuses worker-local scratch storage. The first tree level is written directly from the immutable input slice into a half-size scratch buffer, after which the remaining local levels are performed in place.

This implementation preserves the arithmetic graph derived in Section 1.5; it changes only the scheduling and memory traffic of that graph.

3.6 Processor and Compiler

Table 3 records the platform used for the current measurement campaign. The values are generated from the machine on which this manuscript is built.

Table 3: Processor, compiler, and benchmark environment.	
Item	Configuration
Processor	Apple M1
Physical CPU cores	8
Logical CPU cores	8
Memory	16 GiB
Rust compiler	rustc 1.96.0-nightly (55e86c996 2026-04-02)
Cargo	cargo 1.96.0-nightly (888f67534 2026-03-30)
Operating system	Darwin 25.6.0 arm64
Parallel runtime	Rayon

The benchmarks are compiled in release mode. The repository configuration uses link-time optimization for the performance runs. No result in the paper is taken from a debug build.

3.7 Validation

A simple consistency check is available before examining the final results. The classical Horner method forms one dependency chain, whereas the Boolean-kernel evaluator exposes independent folds at every level. The latter should therefore benefit more strongly from multiple workers once the problem size is large enough to amortize scheduling overhead.

At small sizes the opposite effect is possible: the overhead of creating parallel tasks and managing scratch storage can be comparable to the useful work. The expected behavior is consequently a crossover rather than a uniform speedup at all sizes.

The field backend provides a second validation point. If the specialized Goldilocks reduction has a lower isolated multiplication cost than Montgomery reduction, this advantage should be visible most clearly in the serial Horner benchmark, whose execution is dominated by a chain of multiplications. In the Boolean-kernel evaluator the effect can be reduced or amplified by memory traffic and parallel scheduling. The measurements in the following section are interpreted against these two expectations.

4 Results

Table 4 gives a compact view of the principal measurements. Two experiments are reported separately. The first is the cross-representation sweep, in which all polynomial representations are measured in one benchmark campaign with the same Goldilocks arithmetic backend. The second is the backend experiment, in which specialized Goldilocks arithmetic and Montgomery arithmetic are compared for the same evaluation algorithm. Absolute timings are not compared across these two campaigns.

Table 4: Summary of the main measurements at $N = 2^{20}$. Cross-basis times come from one common benchmark sweep. Backend ratios are the median of five independent same-run comparisons.

Experiment	Quantity	Result
Cross-basis	Boolean-kernel	1.0560 ms
Cross-basis	parallel monomial	1.4116 ms
Cross-basis	kernel / parallel monomial	1.337×
Backend	Montgomery / special, kernel	1.2777×
Backend	Montgomery / special, monomial	1.0591×

The ranking of isolated field multipliers does not determine the ranking of complete evaluators. In the isolated multiplication benchmark, specialized Goldilocks multiplication and Montgomery multiplication have nearly identical central estimates: 1.9107 ns and 1.9162 ns, respectively. Their measurement intervals overlap. No meaningful advantage can therefore be assigned to either method from the isolated multiplication benchmark alone.

The kernel fold behaves differently. The specialized operation

$$t(a + b) + b$$

takes 3.0292 ns in the microbenchmark, compared with 3.7952 ns for the Montgomery implementation. The ratio is 1.2529×. The difference appears only after the multiplication has been embedded in the compound fold operation, where the form of the Goldilocks modulus can be used together with the surrounding additions.

4.1 Field Backend and Complete Evaluators

To test whether the microbenchmark difference survives in the complete evaluator, the four full $N = 2^{20}$ configurations were repeated in five independent Criterion invocations. Table 5 reports the central estimate from each invocation.

Table 5: Repeated full-evaluator measurements at $N = 2^{20}$, in microseconds. M/S denotes Montgomery time divided by specialized Goldilocks time within the same invocation.

Run	K_S	K_M	M_S	M_M	Kernel M/S
1	883.61	1172.40	1453.50	1531.00	1.327
2	989.03	1227.70	1559.40	1654.20	1.241
3	1080.90	1159.90	1537.00	1901.10	1.073
4	1036.00	1323.70	2208.30	1666.00	1.278
5	952.15	1277.60	1607.80	1702.80	1.342
Median	989.03	1227.70	1559.40	1666.00	1.278

The kernel result is stable in sign across all five invocations. The Montgomery implementation is slower in every run, with a median same-run ratio of

$$1.2777 \times .$$

The absolute specialized-kernel estimates vary from $883.61 \mu s$ to $1080.90 \mu s$, which is why the comparison is made within each run rather than by combining absolute times collected at different moments.

The monomial backend comparison is less stable. The same-run ratios range from

$$0.7544 \times \quad \text{to} \quad 1.2369 \times ,$$

and one invocation reverses the ordering. The data therefore do not support a claim that either arithmetic backend is systematically faster for the complete blocked monomial evaluator. This contrasts with the kernel case and suggests that the specialized advantage is tied to the fold structure rather than to scalar field multiplication alone.

4.2 Cross-Representation Evaluation

We next compare the polynomial representations using one common Goldilocks backend. At $N = 2^{20}$, the optimized Boolean-kernel evaluator requires

$$1.0560 \text{ ms},$$

while the parallel monomial implementation requires

$$1.4116 \text{ ms}.$$

The ratio is

$$\frac{1.4116}{1.0560} = 1.337.$$

Equivalently, the Boolean-kernel evaluator uses

$$25.2\%$$

less wall time for this isolated evaluation primitive.

The serial Horner method takes

$$9.0402 \text{ ms},$$

which is

$$8.561 \times$$

the kernel time. Evaluation from the roots-of-unity Lagrange representation is more expensive for an arbitrary off-domain point: the parallel implementation takes

$$17.1990 \text{ ms},$$

and the serial implementation

$$21.3890 \text{ ms}.$$

These numbers characterize only arbitrary single-point evaluation from the given representation; they do not imply that evaluation-form representations are generally inefficient in workloads that operate directly on the evaluation domain.

4.3 Scaling with the Polynomial Size

Figure 3 reports the complete cross-basis sweep from $N = 2^{10}$ to $N = 2^{20}$. The vertical axis is logarithmic in time. The figure shows the complete set of principal algorithms before the individual measurements are interpreted.

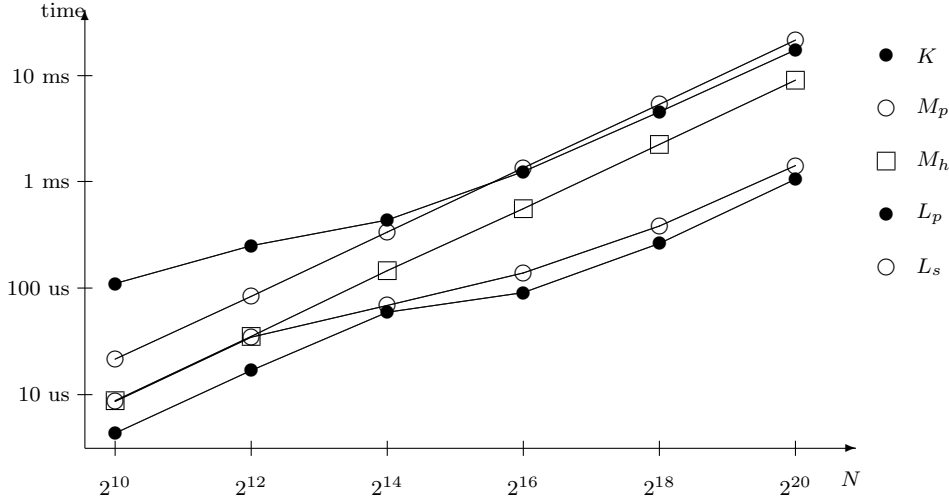


Figure 3: Single-point evaluation time as a function of N for the five implementations in the common cross-basis benchmark. K : optimized Boolean-kernel; M_p : parallel blocked monomial; M_h : serial Horner; L_p : parallel Lagrange; L_s : serial Lagrange. The vertical placement is logarithmic in measured time.

The small-size regime is affected strongly by parallel scheduling and fixed overheads. At $N = 2^{10}$, the kernel evaluator takes approximately half the time of the parallel monomial evaluator, but the absolute difference is only a few microseconds. At $N = 2^{14}$, the two parallel methods become closer, with a ratio of approximately

$$1.169 \times .$$

For larger inputs the separation grows again. At $N = 2^{18}$, the parallel monomial evaluator is

$$1.447 \times$$

the kernel time, and at $N = 2^{20}$ the ratio is

$$1.337 \times .$$

The serial Horner curve grows linearly with a long dependency chain and separates rapidly from the two parallel implementations. The two Lagrange curves have the largest constants because arbitrary off-domain evaluation includes denominator construction, batch inversion, and a full linear accumulation.

4.4 Interpretation

At $N = 2^{20}$, the Boolean-kernel representation gives a lower single-point evaluation time than the parallel monomial baseline in the common benchmark. This difference is not explained by a generally faster scalar field multiplication: isolated Goldilocks and Montgomery multiplication are nearly tied, whereas the specialized Boolean-kernel fold and the complete specialized kernel evaluator show a clear separation.

The result is therefore an interaction between representation and implementation. The Boolean-kernel basis exposes a balanced reduction tree, and the Goldilocks modulus permits the arithmetic inside each fold to be specialized as one compound operation. The benchmark does not establish a corresponding whole-prover speedup. Whether the gain survives in a proof system depends on how long the polynomial can remain in the Boolean-kernel representation and on the cost of any required basis conversion.

5 Conclusions

This article compared single-point evaluation of univariate polynomials in three representations on an Apple M1 processor. The Boolean-kernel basis leads to a balanced reduction tree in which all folds at one level are independent. In the common cross-representation benchmark at $N = 2^{20}$, this evaluator required 1.0560 ms, compared with 1.4116 ms for the blocked parallel monomial implementation. The difference corresponds to 25.2% less wall time for the isolated evaluation operation.

The arithmetic-backend experiment helps identify the source of this difference. Specialized Goldilocks multiplication and Montgomery multiplication have almost identical isolated multiplication times. The fold operation does not: the specialized Goldilocks fold is faster than its Montgomery counterpart, and the same ordering is observed in all five independent runs of the complete kernel evaluator. No similarly stable ordering is observed for the blocked monomial evaluator. The performance difference is therefore associated with the combination of the Boolean-kernel reduction structure and the arithmetic used inside the fold, rather than with a universal advantage of one scalar multiplication routine.

These measurements concern a single processor and an isolated polynomial operation. They do not establish an end-to-end speedup for a proof system. Such a speedup depends on the representation used by the surrounding protocol, the frequency of basis conversion, and the fraction of prover time spent in the evaluated primitive. A monomial-to-kernel conversion requires $O(N \log N)$ field additions, so repeated conversion can erase the benefit measured here.

Further work should therefore address two questions. The first is architectural: the same evaluators should be measured on additional out-of-order CPUs and, where appropriate, on SIMD and accelerator implementations. The second is algorithmic: the Boolean-kernel representation should be integrated into a complete polynomial-commitment or proof-system pipeline in which conversion, commitment, and opening costs are measured together. These experiments would determine whether the single-point advantage observed here persists beyond the isolated evaluation kernel.

References

- [1] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed–solomon interactive oracle proofs of proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics*, pages 14:1–14:17. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018.
- [2] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.
- [3] Jean-Paul Berrut and Lloyd N. Trefethen. Barycentric lagrange interpolation. *SIAM Review*, 46(3):501–517, 2004.
- [4] Timothée Ewart, Francesco Cremonesi, Felix Schürmann, and Fabien Delalondre. Polynomial evaluation on superscalar architecture, applied to the elementary function e^x . *ACM Transactions on Mathematical Software*, 46(3):1–22, 2020.
- [5] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985.